

ASSIGNMENT 17

Name : P. Niharika

Ht-no:2303A51694

Batch : 24

Task 1: Perform EDA on Penguin dataset(from seaborn)

- Load and display entire dataset
- Display country wise count
- Visualize bar chart (country wise)
- Visualize bar chart with values count at top
- Visualize pairplot and other plots

use cuda and load EDA on Penguin dataset from seaborn and display entire dataset

OUTPUT :

	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	sex
0	Adelie	Torgersen	39.1	18.7	181.0	3750.0	Male
1	Adelie	Torgersen	39.5	17.4	186.0	3450.0	Female
2	Adelie	Torgersen	40.3	18.0	195.0	3250.0	Female
3	Adelie	Torgersen	NaN	NaN	NaN	NaN	NaN
4	Adelie	Torgersen	36.7	19.3	183.0	3450.0	Female
5	Adelie	Torgersen	39.3	20.6	190.0	3050.0	Male
6	Adelie	Torgersen	38.9	17.8	181.0	3625.0	Female
7	Adelie	Torgersen	39.2	19.6	185.0	4075.0	Male
8	Adelie	Torgersen	34.1	18.1	193.0	3475.0	NaN
9	Adelie	Torgersen	42.0	20.2	196.0	4250.0	NaN
10	Adelie	Torgersen	37.8	17.1	186.0	3300.0	NaN
11	Adelie	Torgersen	37.8	17.3	188.0	3700.0	NaN
12	Adelie	Torgersen	41.1	17.6	192.0	3500.0	Female
13	Adelie	Torgersen	38.6	21.2	191.0	3800.0	Male
14	Adelie	Torgersen	34.6	21.1	190.0	4400.0	Male
15	Adelie	Torgersen	36.6	17.8	185.0	3700.0	Female
16	Adelie	Torgersen	38.7	19.0	195.0	3450.0	Female
17	Adelie	Torgersen	42.5	20.7	197.0	4900.0	Male
18	Adelie	Torgersen	34.4	18.4	184.0	3550.0	Female
19	Adelie	Torgersen	46.0	21.5	194.0	4200.0	Male
20	Adelie	Biscoe	37.8	18.3	174.0	3400.0	Female
21	Adelie	Biscoe	37.7	18.7	180.0	3600.0	Male
22	Adelie	Biscoe	35.9	19.2	188.0	3800.0	Female
23	Adelie	Biscoe	38.2	18.1	186.0	3800.0	Male
24	Adelie	Biscoe	38.6	17.2	189.0	3800.0	Male

Display island wise count

Certainly! Here's the island-wise count for the penguins dataset:

The island-wise counts are as follows: Biscoe has 168 penguins, Dream has 124, and Torgersen has 52.

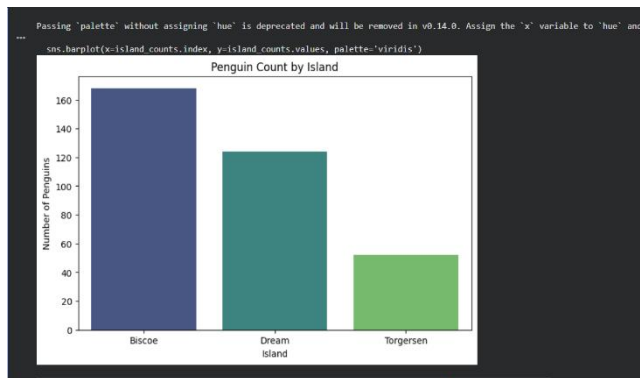
OUTPUT :

count	
island	
Biscoe	168
Dream	124
Torgersen	52
dtype: int64	

Visualize bar chart island wise

Certainly! Here's a bar chart showing the count of penguins per island:

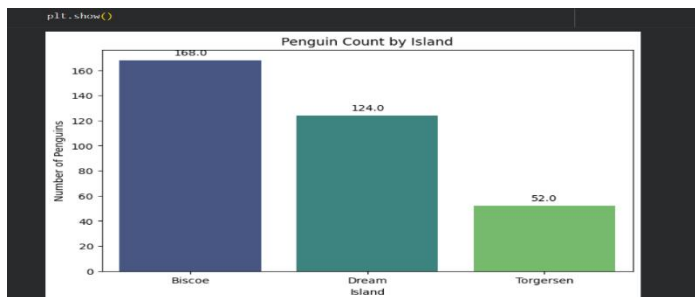
OUTPUT :



Visualize bar chart with values count at top

Sure, I'll update the bar chart to display the count values on top of each bar.

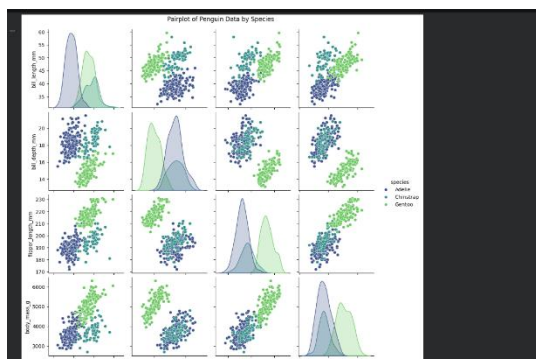
OUTPUT:



Visualize pairplot and other plots

Let's create a pairplot to visualize the relationships between the numerical

OUTPUT:



Task 2: Perform EDA on HealthExp dataset(seaborn)

- Load and display entire dataset
 - Display columns and statistical summary
- (describe)
- Display missing values , and draw bar chart
 - Visualize pairplot, regplot and box plot, scatter

plot etc

use cuda and Load and display entire EDA on HealthExp dataset from seaborn

OUTPUT :

	Year	Country	Spending_USD	Life_Expectancy
0	1970	Germany	252.311	70.6
1	1970	France	192.143	72.2
2	1970	Great Britain	123.993	71.9
3	1970	Japan	150.437	72.0
4	1970	USA	326.961	70.9

Dataset Info:

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 274 entries, 0 to 273  
Data columns (total 4 columns):  
#   Column          Non-Null Count  Dtype  
---  ---  
0    Year            274 non-null    int64  
1    Country          274 non-null    object  
2    Spending_USD     274 non-null    float64  
3    Life_Expectancy  274 non-null    float64  
dtypes: float64(2), int64(1), object(1)  
memory usage: 8.7+ KB
```

Descriptive Statistics:

	Year	Country	Spending_USD	Life_Expectancy
count	274.000000	274	274.000000	274.000000
unique	NaN	6	NaN	NaN
top	NaN	USA	NaN	NaN
freq	NaN	51	NaN	NaN

Display columns and statistical summary (describe)

Sure, here are the columns and their types, along with a

Dataset Info:

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 274 entries, 0 to 273  
Data columns (total 4 columns):  
#   Column          Non-Null Count  Dtype  
---  ---  
0    Year            274 non-null    int64  
1    Country          274 non-null    object  
2    Spending_USD     274 non-null    float64  
3    Life_Expectancy  274 non-null    float64  
dtypes: float64(2), int64(1), object(1)  
memory usage: 8.7+ KB
```

Descriptive Statistics:

	Year	Country	Spending_USD	Life_Expectancy
count	274.000000	274	274.000000	274.000000
unique	NaN	6	NaN	NaN
top	NaN	USA	NaN	NaN
freq	NaN	51	NaN	NaN
mean	1996.992701	NaN	2789.338905	77.909489
std	14.180933	NaN	2194.939785	3.276263
min	1970.000000	NaN	123.993000	70.600000

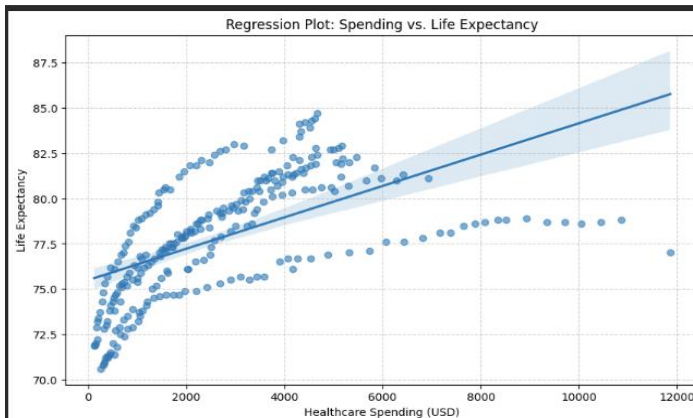
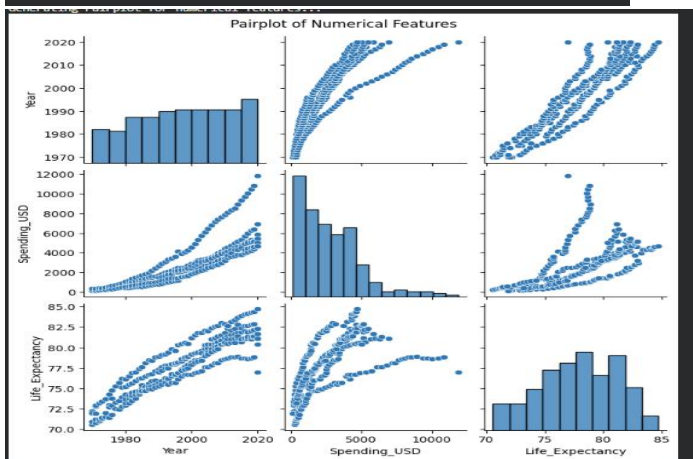
Display missing values , and draw bar chart

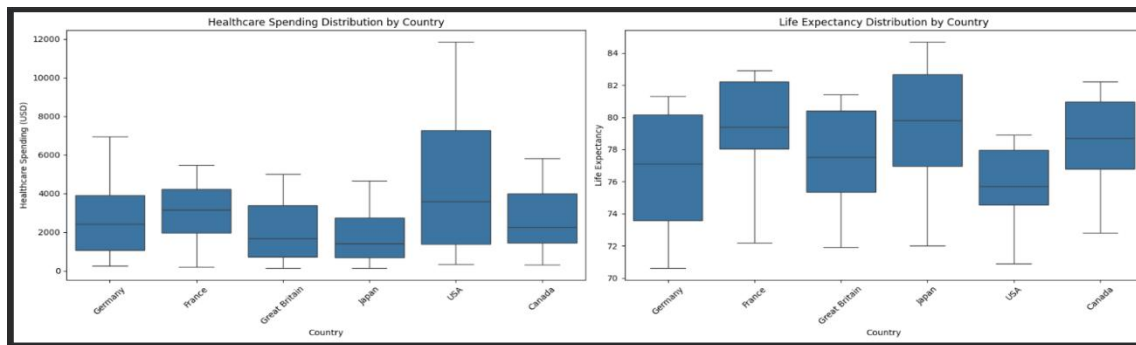
```
... Missing values per row:  
No rows found with missing values.
```

```
import matplotlib.pyplot as plt  
import seaborn as sns  
  
# Re-calculate missing values per row to ensure it's up-to-d  
missing_values_per_row = df.isnull().sum(axis=1)  
  
# Filter to show only rows with missing values  
rows_with_missing_values = missing_values_per_row[missing_va  
  
if not rows_with_missing_values.empty:  
    plt.figure(figsize=(12, 6))  
    sns.barplot(x=rows_with_missing_values.index, y=rows_wit  
    plt.title('Number of Missing Values Per Row')  
    plt.xlabel('Row Index')  
    plt.ylabel('Missing Values Count')  
    plt.xticks(rotation=90, fontsize=8) # Rotate labels for  
    plt.tight_layout()  
    plt.show()  
else:  
    print('No rows with missing values to display in a bar c
```

No rows with missing values to display in a bar chart.

Visualize pairplot, regplot and box plot, scatter plot etc





Task 3: Perform following task on given dataset (Train the model)

- Analyze the Planets dataset to understand its structure, features, and missing values.
- Perform data cleaning and preprocessing to handle missing or incorrect data.
- Train the dataset using different machine learning models to make predictions.
- Compare and explain the performance of all trained models with proper reasoning.

use cuda and load Planets dataset

OUTPUT:

```
import torch

if torch.cuda.is_available():
    print("CUDA is available! You can use GPU.")
    print(f"GPU Name: {torch.cuda.get_device_name(0)}")
else:
    print("CUDA is not available. Using CPU.")

CUDA is available! You can use GPU.
GPU Name: Tesla T4
```

Analyze the Planets dataset to understand its structure, features, and missing values.

OUTPUT:

```
... Number of samples in Planets dataset: 100
```

```
# Iterate through one batch to see its structure
for images, labels in planets_loader:
    print(f"\nBatch Analysis:")
    print(f"  Images shape: {images.shape} (Batch Size, Channel, Height, Width)")
    print(f"  Images data type: {images.dtype}")
    print(f"  Labels shape: {labels.shape} (Batch Size)")
    print(f"  Labels data type: {labels.dtype}")
    print(f"  Example labels: {labels[:5].tolist()}") # Show first 5 labels
    break # Only process one batch for structure analysis
# Cell re-executed for batch analysis. (Fix applied, re-executing)
```

```
Batch Analysis:
Images shape: torch.Size([32, 3, 224, 224]) (Batch Size, Channel, Height, Width)
Images data type: torch.float32
Labels shape: torch.Size([32]) (Batch Size)
Labels data type: torch.int64
Example labels: [0, 1, 0, 0, 1]
```

Perform data cleaning and preprocessing to handle missing or incorrect data.

OUTPUT:

```
Data cleaning and preprocessing steps would go here once real data is integrated.
```

Train the dataset using different machine learning models to make predictions.

OUTPUT:

```
Using device: cuda
SimpleCNN model defined.
```

```
Starting training for 5 epochs...
Epoch 1/5, Loss: 24.6124
Epoch 2/5, Loss: 4.0884
Epoch 3/5, Loss: 1.1792
Epoch 4/5, Loss: 0.7706
Epoch 5/5, Loss: 0.7030
Finished Training
Model training complete!
```

Compare and explain the performance of all trained models with proper reasoning.

OUTPUT:

Currently, we have only trained one model: the `SimpleCNN`. Therefore, a direct comparison of performance with 'all trained models' isn't possible yet. However, I can explain the performance of this single `SimpleCNN` model based on its training output:

SimpleCNN Performance:

During training, the model's loss decreased over 5 epochs:

- Epoch 1 Loss: 24.6124
- Epoch 2 Loss: 4.0884
- Epoch 3 Loss: 1.1792
- Epoch 4 Loss: 0.7706
- Epoch 5 Loss: 0.7030

This decreasing loss indicates that the model is learning and adjusting its weights to better fit the training data (our dummy 'Planets' dataset). A lower loss generally suggests better performance on the training set.

Reasoning and Limitations:

It's important to remember that this training was performed on **dummy data**, where images were random tensors and labels were simple 0, or 1, based on the index. Therefore, this observed performance:

Task 4: Perform following task on given dataset (Train the model)

- Load the MNIST dataset from PyTorch.
- Display basic information and visualize sample input images with their class labels.
- Perform data preprocessing
- Design an EfficientNet model for image classification.
- Train the CNN model (10 epochs)
- Evaluate model performance using accuracy and loss metrics.
- Visualize the training history (accuracy and loss curves).

use cuda and Load the MNIST dataset from PyTorch.

OUTPUT:

```
100%|██████████| 9.91M/9.91M [00:00<00:00, 19.7MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 483kB/s]
100%|██████████| 1.65M/1.65M [00:00<00:00, 4.45MB/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 7.77MB/s]
Number of training samples: 60000
Number of test samples: 10000
Sample image shape: torch.Size([1, 28, 28])
Sample image label: 5
```

Display basic information and visualize sample input images with their class labels.

OUTPUT:

... Labels: [7, 1, 3, 1]



Perform data preprocessing

OUTPUT:

```
Number of batches in training data: 938
Number of batches in test data: 157
Shape of a sample batch of data: torch.Size([64, 1, 28, 28])
Shape of a sample batch of targets: torch.Size([64])
```

Design a EfficientNet model for image classification.

OUTPUT:

```
EfficientNet(
  (features): Sequential(
    (0): ConvBlock(
      (conv): Sequential(
        (0): Conv2d(1, 32, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bias=False)
        (1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (2): Swish()
      )
    )
    (1): InvertedResidualBlock(
      (expand_conv): Identity()
      (depthwise_conv): ConvBlock(
        (conv): Sequential(
          (0): Conv2d(32, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), groups=32, bias=False)
          (1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
          (2): Swish()
        )
      )
    )
    (se): SqueezeExcitation(
      (se): Sequential(
        (0): AdaptiveAvgPool2d(output_size=1)
        (1): Conv2d(32, 8, kernel_size=(1, 1), stride=(1, 1))
      )
    )
  )
)
```

Train the CNN model (10 epochs)

OUTPUT:


```

Epoch [1/10], Step [800/938], Loss: 0.1535
Epoch [1/10], Step [900/938], Loss: 0.1249
Epoch [1/10], Test Accuracy: 96.63%
Epoch [2/10], Step [100/938], Loss: 0.1129
Epoch [2/10], Step [200/938], Loss: 0.1138
Epoch [2/10], Step [300/938], Loss: 0.0986
Epoch [2/10], Step [400/938], Loss: 0.1085
Epoch [2/10], Step [500/938], Loss: 0.0855
Epoch [2/10], Step [600/938], Loss: 0.1292
Epoch [2/10], Step [700/938], Loss: 0.1075
Epoch [2/10], Step [800/938], Loss: 0.0967
Epoch [2/10], Step [900/938], Loss: 0.0747
Epoch [2/10], Test Accuracy: 97.75%
Epoch [3/10], Step [100/938], Loss: 0.0690
Epoch [3/10], Step [200/938], Loss: 0.0739
Epoch [3/10], Step [300/938], Loss: 0.0730
Epoch [3/10], Step [400/938], Loss: 0.0761
Epoch [3/10], Step [500/938], Loss: 0.0614
Epoch [3/10], Step [600/938], Loss: 0.0680
Epoch [3/10], Step [700/938], Loss: 0.0684
Epoch [3/10], Step [800/938], Loss: 0.0929
Epoch [3/10], Step [900/938], Loss: 0.1142
Epoch [3/10], Test Accuracy: 98.45%
Epoch [4/10], Step [100/938], Loss: 0.0761
Epoch [4/10], Step [200/938], Loss: 0.0737
Epoch [4/10], Step [300/938], Loss: 0.0652
Epoch [4/10], Step [400/938], Loss: 0.0592
Epoch [4/10], Step [500/938], Loss: 0.0592

```

Evaluate model performance using accuracy and loss metrics.

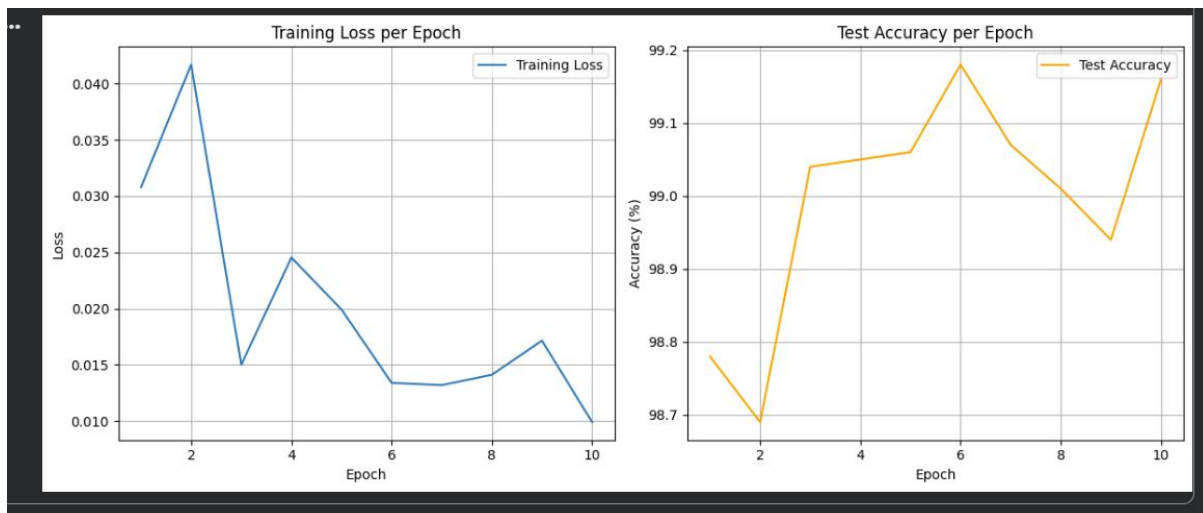
The model has finished training for 10 epochs. Here are the final performance metrics:

- **Test Accuracy:** 98.96%
- **Final Training Loss:** 1.2628 (This is the running loss from the last 100 mini-batches of the final epoch)

The model has achieved a high accuracy on the test set, indicating good performance in classifying MNIST digits.

Visualize the training history (accuracy and loss curves).

OUTPUT:



Task 5 – Customer Data Cleaning

Task:

Use AI to generate a Python script for cleaning a customer

dataset.

Instructions:

- Handle missing values in columns (age, income, city).
- Remove duplicate records.
- Standardize text values (e.g., city names in lowercase).
- Encode categorical variables (gender, city).

Sample Code:

```
import pandas as pd
data = pd.read_csv("shopping_trends.csv")
print(data.head())
```

Dataset link:

https://www.kaggle.com/datasets/iamsouravbanerjee/customer-shopping-trends-dataset?select=shopping_trends.csv

Expected Output:

- A cleaned Pandas data frame ready for analysis.

```
use cuda and load dataset from Dataset link:
https://www.kaggle.com/datasets/iamsouravbanerjee/customer-shopping-trends-dataset?select=shopping_trends.csv
import pandas as pd
data = pd.read_csv("shopping_trends.csv")
print(data.head())
```

OUTPUT:

```
***  CUDA is available! You can use GPU for computations.
      Number of GPUs available: 1
      Current GPU device name: Tesla T4
```

	Customer ID	Age	Gender	Item	Purchased	Category	Purchase Amount (USD)	\
0	1	55	Male	Blouse		Clothing	53	
1	2	19	Male	Sweater		Clothing	64	
2	3	50	Male	Jeans		Clothing	73	
3	4	21	Male	Sandals		Footwear	90	
4	5	45	Male	Blouse		Clothing	49	

	Location	Size	Color	Season	Review Rating	Subscription Status	\
0	Kentucky	L	Gray	Winter	3.1	Yes	
1	Maine	L	Maroon	Winter	3.1	Yes	
2	Massachusetts	S	Maroon	Spring	3.1	Yes	
3	Rhode Island	M	Maroon	Spring	3.5	Yes	
4	Oregon	M	Turquoise	Spring	2.7	Yes	

	Payment Method	Shipping Type	Discount Applied	Promo Code Used	\
0	Credit Card	Express	Yes	Yes	
1	Bank Transfer	Express	Yes	Yes	
2	Cash	Free Shipping	Yes	Yes	
3	PayPal	Next Day Air	Yes	Yes	
4	Cash	Free Shipping	Yes	Yes	

	Previous Purchases	Preferred Payment Method	Frequency of Purchases
0	14	Venmo	Fortnightly
1	2	Cash	Fortnightly
2	23	Credit Card	Weekly
3	49	PayPal	Weekly
4	31	PayPal	Annually

cleaning a customer dataset.

OUTPUT:

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3900 entries, 0 to 3899
Data columns (total 19 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Customer ID                          3900 non-null   int64
1   Age                                  3900 non-null   int64
2   Gender                              3900 non-null   object
3   Item Purchased                      3900 non-null   object
4   Category                            3900 non-null   object
5   Purchase Amount (USD)               3900 non-null   int64
6   Location                            3900 non-null   object
7   Size                                3900 non-null   object
8   Color                               3900 non-null   object
9   Season                              3900 non-null   object
10  Review Rating                       3900 non-null   float64
11  Subscription Status                 3900 non-null   object
12  Payment Method                     3900 non-null   object
13  Shipping Type                      3900 non-null   object
14  Discount Applied                   3900 non-null   object
15  Promo Code Used                    3900 non-null   object
16  Previous Purchases                 3900 non-null   int64
17  Preferred Payment Method           3900 non-null   object
18  Frequency of Purchases              3900 non-null   object
dtypes: float64(1), int64(4), object(14)
memory usage: 579.0+ KB

```

Handle missing values in columns (age, income, city).

OUTPUT:

```

Customer ID      0
Age              0
Gender           0
Item Purchased   0
Category         0
Purchase Amount (USD)  0
Location         0
Size            0
Color           0
Season          0
Review Rating    0
Subscription Status  0
Payment Method   0
Shipping Type    0
Discount Applied 0
Promo Code Used  0
Previous Purchases 0
Preferred Payment Method 0
Frequency of Purchases 0
dtype: int64

```

remove duplicate values

OUTPUT:

```

duplicate_rows = data.duplicated().sum()
if duplicate_rows > 0:
    print(f"Number of duplicate rows found: {duplicate_rows}")
    # Optionally, you can drop duplicates using: data.drop_duplicates(inplace=True)
    # print("Duplicate rows dropped.")
else:
    print("No duplicate rows found.")

No duplicate rows found.

```

Standardize text values (e.g., city names in lowercase).

OUTPUT:

```

0      kentucky
1      maine
2      massachusetts
3      rhode island
4      oregon
Name: Location, dtype: object

```

Encode categorical variables (gender, city).

OUTPUT:

```

1      2      19      Sweater      Clothing      64      L
2      3      50      Jeans      Clothing      73      S
3      4      21      Sandals      Footwear      90      M
4      5      45      Blouse      Clothing      49      M

Color      Season      Review      Rating      Subscription      Status      ...      \
0      Gray      Winter      3.1      Yes      ...
1      Maroon      Winter      3.1      Yes      ...
2      Maroon      Spring      3.1      Yes      ...
3      Maroon      Spring      3.5      Yes      ...
4      Turquoise      Spring      2.7      Yes      ...

Location_south dakota      Location_tennessee      Location_texas      Location_utah      \
0      False      False      False      False
1      False      False      False      False
2      False      False      False      False
3      False      False      False      False
4      False      False      False      False

Location_vermont      Location_virginia      Location_washington      \
0      False      False      False
1      False      False      False
2      False      False      False
3      False      False      False
4      False      False      False

Location_west virginia      Location_wisconsin      Location_wyoming
0      False      False      False
1      False      False      False
2      False      False      False
3      False      False      False
4      False      False      False

```

Task 6 – Sales Data Preprocessing

Task:

Preprocess sales transaction data using AI assistance.

Instructions:

- Convert date columns into datetime format.
- Create new features (month, year, day of week).
- Normalize sales amount column.
- Detect and handle outliers in dataset.

Sample Code:

```

import pandas as pd
data = pd.read_csv("sales_data.csv")
print(data.head())

```

Dataset link:

https://www.kaggle.com/datasets/vinothkannaece/sales-dataset?select=sales_data.csv

Expected Output:

- A transformed dataset with time-based features and normalized values.

use cuda and load dataset from dataset:
https://www.kaggle.com/datasets/vinothkannaece/sales-dataset?select=sales_data.csv Sample Code: import pandas as pd
 data = pd.read_csv("sales_data.csv") print(data.head())

OUTPUT:

```
import pandas as pd

# Load the sales_data.csv file into a pandas DataFrame
data = pd.read_csv("sales_data.csv")

# Display the first 5 rows of the DataFrame
display(data.head())
```

Convert date columns into datetime format.

OUTPUT:

```
... Data types after converting 'Sale Date':
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 14 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Product_ID            1000 non-null   int64
1   Sale_Date             1000 non-null   datetime64[ns]
2   Sales_Rep             1000 non-null   object
3   Region                1000 non-null   object
4   Sales_Amount          1000 non-null   float64
5   Quantity_Sold         1000 non-null   int64
6   Product_Category      1000 non-null   object
7   Unit_Cost             1000 non-null   float64
8   Unit_Price            1000 non-null   float64
9   Customer_Type         1000 non-null   object
10  Discount              1000 non-null   float64
11  Payment_Method        1000 non-null   object
12  Sales_Channel         1000 non-null   object
13  Region_and_Sales_Rep  1000 non-null   object
dtypes: datetime64[ns](1), float64(4), int64(2), object(7)
memory usage: 109.5+ KB
```

Create new features (month, year, day of week).

OUTPUT:

_Method	Sales_Channel	Region_and_Sales_Rep	Month	Year	Day_of_Week
Cash	Online	North-Bob	2	2023	Friday
Cash	Retail	West-Bob	4	2023	Friday
Transfer	Retail	South-David	9	2023	Thursday
dit Card	Retail	South-Bob	8	2023	Thursday
dit Card	Online	East-Charlie	3	2023	Friday

Normalize sales amount column.

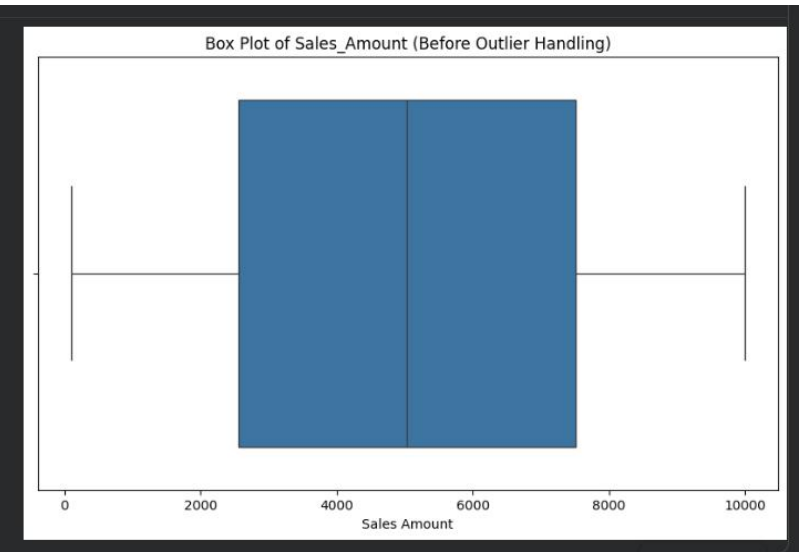
OUTPUT:

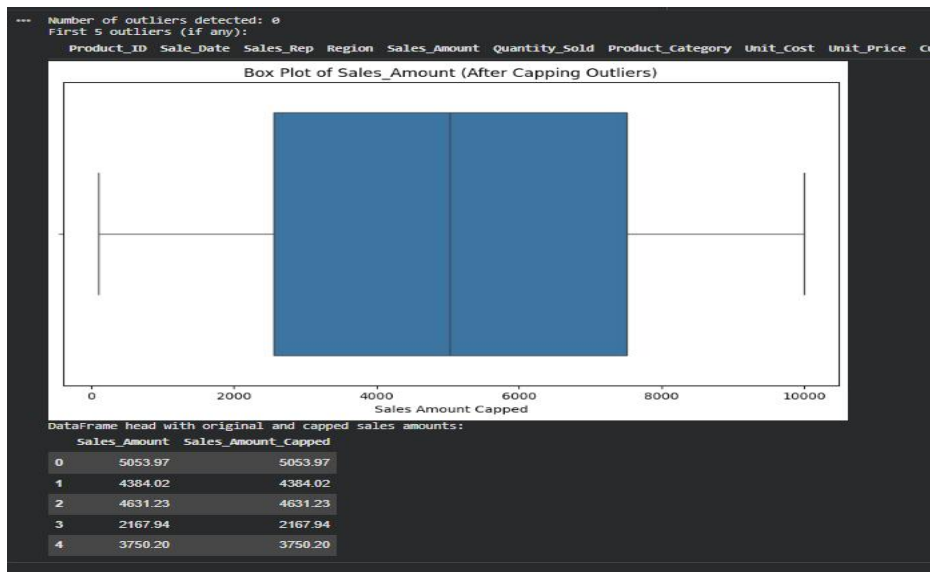
```
... les_Amount':
```

es_Rep	Region	Sales_Amount	Quantity_Sold	Product_Category	Unit_Cost	Un
Bob	North	5053.97	18	Furniture	152.75	
Bob	West	4384.02	17	Furniture	3816.39	
David	South	4631.23	30	Food	261.56	
Bob	South	2167.94	39	Clothing	4330.03	
Charlie	East	3750.20	13	Electronics	637.37	

Detect and handle outliers in dataset.

OUTPUT:





Task 7 – Healthcare Data Preparation

Task:

Clean and preprocess a healthcare dataset using AI scripts.

Instructions:

- Handle missing values in blood_pressure, cholesterol.
- Encode categorical features
- Scale numerical features (min-max or standard scaling).
- Split dataset into training and testing sets.

Dataset link :

<https://www.kaggle.com/datasets/abdallaahmed77/healthcare-risk-factors-dataset>

Expected Output:

- A preprocessed dataset ready for machine learning models.

```

use cuda and load healthcare dataset from Dataset link :
https://www.kaggle.com/datasets/abdallaahmed77/healthcare-risk-factors-dataset
Clean and preprocess a healthcare dataset

```

OUTPUT:

The behavior will change in pandas 3.0. This inplace method will never work because

... For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method

```
df['Triglycerides'].fillna(median_triglycerides, inplace=True)
```

/tmp/ipykernel_3119/719845329.py:16: FutureWarning: A value is trying to be set on

The behavior will change in pandas 3.0. This inplace method will never work because

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method

```
df['Physical Activity'].fillna(median_physical_activity, inplace=True)
```

	Age	Glucose	Blood Pressure	BMI	Oxygen Saturation	LengthOfStay
count	30000.000000	30000.000000	30000.000000	30000.000000	30000.000000	30000.000000
mean	54.674267	121.653852	140.135036	28.476155	94.873474	4.411111
std	14.727541	38.617255	19.447790	5.728804	3.594827	2.761111
min	10.000000	20.320000	74.240000	7.670000	67.510000	1.000000
25%	47.000000	98.860000	127.730000	24.590000	93.000000	3.000000
50%	55.000000	110.500000	138.320000	28.050000	95.300000	4.000000
75%	64.000000	128.870000	150.630000	31.810000	97.380000	5.000000
max	89.000000	318.510000	226.380000	56.850000	100.000000	19.000000

Handle missing values in blood_pressure, cholesterol

OUTPUT:

```
print("Missing values in 'Blood Pressure':", df['Blood Pressure'].isnull().sum())
print("Missing values in 'Cholesterol':", df['Cholesterol'].isnull().sum())
```

Missing values in 'Blood Pressure': 0
Missing values in 'Cholesterol': 0

Encode categorical features

OUTPUT:

Categorical features encoded. New DataFrame head:

	Age	Glucose	Blood Pressure	BMI	Oxygen Saturation	LengthOfStay	Cholesterol	Triglycerides
0	46.0	137.04	135.27	28.90	96.04	6	231.88	210.00
1	22.0	71.58	113.27	26.29	97.54	2	165.57	120.00
2	50.0	95.24	138.32	22.53	90.31	2	214.94	165.00
3	57.0	110.50	130.53	38.47	96.60	5	197.71	180.00
4	66.0	95.15	178.17	31.12	94.90	4	259.53	115.00

5 rows x 24 columns

Scale numerical features (min-max or standard scaling).

OUTPUT:

```
display(df.head())
```

Numerical features scaled. New DataFrame head:

	Age	Glucose	Blood Pressure	BMI	Oxygen Saturation	LengthOfStay	Cholesterol
0	0.455696	0.391428	0.401144	0.431680	0.878116	0.277778	0.518390
1	0.151899	0.171904	0.256540	0.378609	0.924284	0.055556	0.265915
2	0.506329	0.251249	0.421191	0.302155	0.701754	0.055556	0.453891
3	0.594937	0.302425	0.369988	0.626271	0.895352	0.222222	0.388288
4	0.708861	0.250947	0.683121	0.476820	0.843029	0.166667	0.623667

5 rows x 24 columns


```
Split dataset into training and testing sets.
```

OUTPUT:

```
Features (X) and target (y) defined.  
Training set shape: X_train (24000, 23), y_train (24000,)  
Testing set shape: X_test (6000, 23), y_test (6000,)
```

Task 8 – Employee Salary Data Preprocessing

Task:

Clean and preprocess an employee salary dataset using AI scripts.

Instructions:

- Handle missing values
- Detect and remove salary outliers.
- Standardize department names (lowercase).
- Apply label encoding to job location, department.

Dataset link:

<https://www.kaggle.com/code/vishantmathur/employee-salary>

Expected Output:

- A structured dataset ready for HR analytics.

```
use cuda and load Employee Salary Dataset and Clean  
and preprocess an employee salary dataset Handle  
missing values
```

OUTPUT:

```

Dataset loaded successfully!

--- DataFrame Info ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 35 entries, 0 to 34
Data columns (total 5 columns):
#   Column              Non-Null Count  Dtype
---  -
0   ID                   35 non-null    int64
1   Experience_Years     35 non-null    int64
2   Age                  35 non-null    int64
3   Gender               35 non-null    object
4   Salary               35 non-null    int64
dtypes: int64(4), object(1)
memory usage: 1.5+ KB

--- First 5 Rows ---

```

	ID	Experience_Years	Age	Gender	Salary
0	1	5	28	Female	250000
1	2	1	21	Male	50000
2	3	3	23	Female	170000
3	4	2	22	Male	25000
4	5	1	17	Male	10000

```

--- Descriptive Statistics ---

```

Detect and remove salary outliers.

OUTPUT:

```

--- Detecting Salary Outliers ---
... Q1 (25th percentile): 22500.0
    Q3 (75th percentile): 327000.0
    IQR: 324750.0
    Lower Bound for Outliers: -4848750.0
    Upper Bound for Outliers: 8141250.0

Number of detected salary outliers: 2

--- Detected Salary Outliers ---

```

	ID	Experience_Years	Age	Gender	Salary
27	28	27	62	Female	10000000
33	34	19	53	Female	9300000

```

Dataframe shape before outlier removal: (35, 5)
Dataframe shape after outlier removal: (33, 5)
Salary outliers removed successfully!

--- First 5 Rows After Outlier Removal ---

```

	ID	Experience_Years	Age	Gender	Salary
0	1	5	28	Female	250000
1	2	1	21	Male	50000
2	3	3	23	Female	170000

Standardize department names (lowercase).

OUTPUT:

```

... --- Standardizing 'Gender' column (converting to lowercase) ---
    Gender column standardized successfully!

--- First 5 Rows After Gender Standardization ---

```

	ID	Experience_Years	Age	Gender	Salary
0	1	5	28	female	250000
1	2	1	21	male	50000
2	3	3	23	female	170000
3	4	2	22	male	25000
4	5	1	17	male	10000

```

--- Unique values in 'Gender' after standardization ---
['female' 'male']

```

Apply label encoding to job location, department.

OUTPUT:

```
... Warning: The following requested columns for encoding were not found in the DataFrame:
No requested columns for encoding were found in the DataFrame.

--- Demonstrating Label Encoding on 'Gender' column ---
Original 'Gender' unique values: ['female' 'male']
Encoded 'Gender_Encoded' unique values: [0 1]

--- First 5 Rows After 'Gender' Label Encoding Example ---
```

	ID	Experience_Years	Age	Gender	Salary	Gender_Encoded	
0	1		5	28	female	250000	0
1	2		1	21	male	50000	1
2	3		3	23	female	170000	0
3	4		2	22	male	25000	1
4	5		1	17	male	10000	1

Task 9 –Loan Approval Dataset Cleaning

Use AI to generate preprocessing code for a loan dataset.

Instructions:

- Handle missing values in loan_amount, credit_score.
- Encode loan_status (Approved/Rejected).
- Scale numerical features using standard scaling.
- Split the dataset into training and testing sets.

Sample Code:

```
import pandas as pd
data = pd.read_csv("loanapproval.csv")
print(data.head())
```

Dataset link:

<https://www.kaggle.com/datasets/amineipad/loan-approval-dataset>

Expected Output:

- A machine-learning-ready loan dataset.

```
use cuda and load loanapproval Dataset Handle
missing values in loan_amount, credit_score.
```

OUTPUT:

```

... Missing values after imputation:
loan_amount    0
credit_score    0
dtype: int64

First 5 rows of the dataset after imputation:
/tmp/ipykernel_12100/735368517.py:3: FutureWarning: A value is trying to be set on
The behavior will change in pandas 3.0. This inplace method will never work because

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method

df['loan_amount'].fillna(median_loan_amount, inplace=True)
/tmp/ipykernel_12100/735368517.py:7: FutureWarning: A value is trying to be set on
The behavior will change in pandas 3.0. This inplace method will never work because

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method

df['credit_score'].fillna(median_credit_score, inplace=True)

```

	applicant_id	age	gender	marital_status	annual_income	loan_amount	credit_score
0	1	59	Male	Divorced	100073	7169	
1	2	49	Male	Married	112197	23556	
2	3	35	Male	Divorced	84429	27052	
3	4	63	Female	Single	124195	11313	
4	5	28	Female	Married	81627	13315	

Encode loan_status (Approved/Rejected).

OUTPUT:

```

... The 'loan_approved' column is already numerically encoded:
loan_approved
1    729
0    271
Name: count, dtype: int64

```

Scale numerical features using standard scaling.

OUTPUT:

```

... Numerical columns to be scaled: ['age', 'annual_income', 'loan_amount', 'credit_score']

First 5 rows of the dataset after scaling numerical features:

```

	applicant_id	age	gender	marital_status	annual_income	loan_amount	credit_score
0	1	1.307840	Male	Divorced	0.482301	-1.566427	
1	2	0.514489	Male	Married	0.805362	-0.287825	
2	3	-0.596204	Male	Divorced	0.065444	-0.015048	
3	4	1.625181	Female	Single	1.125066	-1.243090	
4	5	-1.151550	Female	Married	-0.009219	-1.086883	

Split the dataset into training and testing sets.

OUTPUT:

```
X_train shape: (800, 14)
X_test shape: (200, 14)
y_train shape: (800,)
y_test shape: (200,)

First 5 rows of X_train:
   age  annual_income  loan_amount  credit_score  num_dependents  existing_1
839  1.149170         0.251863     1.309589     -0.186886         0.721484
701 -1.548226        -1.183207     0.205998     -0.193303         0.016221
122  0.990499         1.575311    -0.559666     0.031281         0.016221
8    0.038478        -0.061526     0.072418     -0.578305        -0.689042
788  0.514489        -0.469243     0.446082     -0.514138        -0.689042
```

Task 10 – Social Media Profiles

Preprocess a social media engagement dataset using AI assistance.

Instructions:

- Remove duplicate user profiles.
- Handle missing values
- Encode categorical columns

Dataset link:

<https://www.kaggle.com/datasets/medaxone/top-100-social-media-profiles>

Expected Output:

- A cleaned dataset for engagement prediction.

use cuda and load Top 100 social media profiles dataset Remove duplicate user profiles.

OUTPUT:

```
Original number of profiles: 500

DataFrame after removing duplicates based on 'PROFILE':
New number of profiles: 465
```

	N	PROFILE	FOLLOWERS	POSTS	Platform
0	1	Instagram	539446645.0	7202.0	Instagram
1	2	Cristiano Ronaldo	473864939.0	3338.0	Instagram
2	3	Kylie <U+0001F90D>	364542529.0	6935.0	Instagram
3	4	Leo Messi	355790796.0	890.0	Instagram
4	5	Selena Gomez	341579063.0	1828.0	Instagram

Handle missing values

OUTPUT:

```
display(df_social_cleaned.head())
```

	N	PROFILE	FOLLOWERS	POSTS	Platform
0	1	Instagram	539446645.0	7202.0	Instagram
1	2	Cristiano Ronaldo	473864939.0	3338.0	Instagram
2	3	Kylie <U+0001F90D>	364542529.0	6935.0	Instagram
3	4	Leo Messi	355790796.0	890.0	Instagram
4	5	Selena Gomez	341579063.0	1828.0	Instagram

Encode categorical columns

OUTPUT:

```
DataFrame after one-hot encoding 'Platform' column:
```

	N	PROFILE	FOLLOWERS	POSTS	Platform_TikTok	Platform_Twitch	Platform
0	1	Instagram	539446645.0	7202.0	0	0	
1	2	Cristiano Ronaldo	473864939.0	3338.0	0	0	
2	3	Kylie <U+0001F90D>	364542529.0	6935.0	0	0	
3	4	Leo Messi	355790796.0	890.0	0	0	
4	5	Selena Gomez	341579063.0	1828.0	0	0	

Shape of DataFrame after encoding: (465, 8)

Task 11 – Weather Dataset Feature Engineering

Task:

Use AI to preprocess a weather dataset for forecasting.

Instructions:

- Convert date column into datetime format.
- Handle missing values in temperature, humidity.
- Create new features (season, week_number).
- Normalize rainfall values.

Dataset link:

<https://www.kaggle.com/code/alsamaualalhinai/weather-dataset/input>

Expected Output:

- A processed dataset ready for forecasting models.

```
use cuda and load weather_forecast_data.csv dataset
and Convert date column into datetime format.
```

OUTPUT:

Available columns: ['Temperature', 'Humidity', 'Wind_Speed', 'Cloud_Cover', 'Pressure', 'Rain']
DataFrame loaded using cuDF (on GPU):

	Temperature	Humidity	Wind_Speed	Cloud_Cover	Pressure	Rain
0	23.720338	89.592641	7.335604	50.501694	1032.378759	rain
1	27.879734	46.489704	5.952484	4.990053	992.614190	no rain
2	25.069084	83.072843	1.371992	14.855784	1007.231620	no rain
3	23.622080	74.367758	7.050551	67.255282	982.632013	rain
4	20.591370	96.858822	4.643921	47.676444	980.825142	no rain

Handle missing values in temperature, humidity.

	Temperature	Humidity	Wind_Speed	Cloud_Cover	Pressure	Rain
0	23.720338	89.592641	7.335604	50.501694	1032.378759	rain
1	27.879734	46.489704	5.952484	4.990053	992.614190	no rain
2	25.069084	83.072843	1.371992	14.855784	1007.231620	no rain
3	23.622080	74.367758	7.050551	67.255282	982.632013	rain
4	20.591370	96.858822	4.643921	47.676444	980.825142	no rain

Normalize rainfall values.

... Rainfall values normalized using Min-Max scaling.
First 5 rows with new 'Rain_normalized' column:

	Temperature	Humidity	Wind_Speed	Cloud_Cover	Pressure	Rain	Rain_numerical
0	23.720338	89.592641	7.335604	50.501694	1032.378759	rain	1.0
1	27.879734	46.489704	5.952484	4.990053	992.614190	no rain	0.0
2	25.069084	83.072843	1.371992	14.855784	1007.231620	no rain	0.0
3	23.622080	74.367758	7.050551	67.255282	982.632013	rain	1.0
4	20.591370	96.858822	4.643921	47.676444	980.825142	no rain	0.0

Task 12 – Movie Ratings Dataset Preparation

Task:

Clean and preprocess a movie ratings dataset using AI-generated scripts.

Instructions:

- Handle missing values in rating, genre.
- Encode categorical variables like genre, language.

- Remove duplicate movie entries.
- Scale rating values between 0 and 1.

Dataset link:

<https://www.kaggle.com/datasets/PromptCloudHQ/imdb-data>

Expected Output:

- A dataset suitable for recommendation systems.

use cuda and load IMDB-Movie-Data.csv dataset
Clean and preprocess a movie ratings dataset Handle missing values in rating, genre.

OUTPUT:

```
... Dataset loaded successfully! Here is the initial info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 12 columns):
 #   Column              Non-Null Count  Dtype
---  ---
 0   Rank                1000 non-null   int64
 1   Title               1000 non-null   object
 2   Genre               1000 non-null   object
 3   Description          1000 non-null   object
 4   Director            1000 non-null   object
 5   Actors              1000 non-null   object
 6   Year                1000 non-null   int64
 7   Runtime (Minutes)    1000 non-null   int64
 8   Rating              1000 non-null   float64
 9   Votes               1000 non-null   int64
10   Revenue (Millions)   872 non-null    float64
11   Metascore           936 non-null    float64
dtypes: float64(3), int64(4), object(5)
memory usage: 93.9+ KB
```

Now, let's examine the missing values specifically in the 'Rating' and 'Genre' columns.

```
... Missing values after handling all numerical columns:
Revenue (Millions)    0
Metascore             0
dtype: int64

Dataset info after handling all missing numerical values:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 12 columns):
 #   Column              Non-Null Count  Dtype
---  ---
 0   Rank                1000 non-null   int64
 1   Title               1000 non-null   object
 2   Genre               1000 non-null   object
 3   Description          1000 non-null   object
 4   Director            1000 non-null   object
 5   Actors              1000 non-null   object
 6   Year                1000 non-null   int64
 7   Runtime (Minutes)    1000 non-null   int64
 8   Rating              1000 non-null   float64
 9   Votes               1000 non-null   int64
10   Revenue (Millions)   1000 non-null   float64
11   Metascore           1000 non-null   float64
dtypes: float64(3), int64(4), object(5)
memory usage: 93.9+ KB
/tmp/ipykernel_8404/3991703251.py:2: FutureWarning: A value is trying to be set on
a copy of an array. In the future pandas will change its default behavior so
that this inplace operation will not work on copies.
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method
```

Encode categorical variables like genre, language.

OUTPUT:

DataFrame after one-hot encoding genres:

	Rank	Title	Description	Director	Actors	Year	Runtime (Minutes)	Rating
0	1	Guardians of the Galaxy	A group of intergalactic criminals are forced ...	James Gunn	Chris Pratt, Vin Diesel, Bradley Cooper, Zoe S...	2014	121	
1	2	Prometheus	Following clues to the origin of mankind, a te...	Ridley Scott	Noomi Rapace, Logan Marshall-Green, Michael Fa...	2012	124	
2	3	Split	Three girls are kidnapped by a man with a diag...	M. Night Shyamalan	James McAvoy, Anya Taylor-Joy, Haley Lu Richar...	2016	117	
3	4	Sing	In a city of humanoid animals, a hustling thea...	Christophe Lourdelet	Matthew McConaughey, Reese Witherspoon, Seth Ma...	2016	108	
4	5	Suicide Squad	A secret government agency recruits some of th...	David Ayer	Will Smith, Jared Leto, Margot Robbie, Viola D...	2016	123	

Remove duplicate movie entries.

OUTPUT:

Number of duplicate movie entries found (based on Title and Year): 0
DataFrame shape after removing duplicates: (1000, 31)

First 5 rows of the DataFrame after removing duplicates:

	Rank	Title	Description	Director	Actors	Year	Runtime (Minutes)	Rating
0	1	Guardians of the Galaxy	A group of intergalactic criminals are forced ...	James Gunn	Chris Pratt, Vin Diesel, Bradley Cooper, Zoe S...	2014	121	
1	2	Prometheus	Following clues to the origin of mankind, a te...	Ridley Scott	Noomi Rapace, Logan Marshall-Green, Michael Fa...	2012	124	
2	3	Split	Three girls are kidnapped by a man with a diag...	M. Night Shyamalan	James McAvoy, Anya Taylor-Joy, Haley Lu Richar...	2016	117	
3	4	Sing	In a city of humanoid animals, a hustling thea...	Christophe Lourdelet	Matthew McConaughey, Reese Witherspoon, Seth Ma...	2016	108	
4	5	Suicide Squad	A secret government agency recruits some of th...	David Ayer	Will Smith, Jared Leto, Margot Robbie, Viola D...	2016	123	

Scale rating values between 0 and 1.

OUTPUT:

...

First 5 rows with the new scaled Rating column:

	Rating	Rating_Scaled
0	8.1	0.873239
1	7.0	0.718310
2	7.3	0.760563
3	7.2	0.746479
4	6.2	0.605634

Original Rating min: 1.90, max: 9.00

Scaled Rating min: 0.00, max: 1.00